AGE 219: Introduction to Computer Programming using Python - Assignments

This document was designed to build your understanding of Python programming. The questions will often refer to the following conceptual 5 digital tech artifacts:

- **Number Guesser Game:** CLI game for guessing a number.
- **GradeBook Analyzer:** Script for analyzing student grades.
- **Headline Fetcher:** Tool for getting news headlines.
- **TaskMaster CLI:** Command-line to-do list.
- **QuickCalc:** Basic command-line calculator.

Assignment 1: Introduction to Python and Basic Syntax

1. What is Python, and why is it considered a high-level programming language?
2. Explain the difference between a compiler and an interpreter. Which does Python primarily use?
3. Write a Python script to print the string "Hello, AGE 219!".
4. What is a variable in Python? Create a variable course_name and assign it the value "Introduction to Computer Programming".
5. List three common data types in Python and provide an example for each.
6. What is a syntax error? Give an example of Python code that would produce a syntax error.
7. How do you write a single-line comment in Python? How do you write a multi-line comment?
8. Create two variables, num1 and num2, assign them integer values, and print their sum.
9. What is the output of the following code? print("Python" + " " + "is" + " " + "fun")
10. Explain the purpose of the print() function.
11. If you were building the **QuickCalc** artifact, what data type would you use to store the numbers entered by the user for calculation? Why?
12. For the **Number Guesser Game**, if the secret number is 7, write a Python statement to store this in a variable.
13. What are keywords in Python? List two examples.
14. Is Python case-sensitive? Explain with an example.
15. How can you find the data type of a variable in Python? Write a code snippet to demonstrate.

Assignment 2: Operators and Input/Output

1. List the arithmetic operators available in Python.
2. What is the difference between the / and // operators in Python? Provide an

example.

3. Write a Python script that asks the user for their name and then prints "Hello, [name]".
4. Explain the purpose of the input() function. What data type does it return by default?
5. What are assignment operators? Give an example of a compound assignment operator (e.g., +=).
6. Write a script that takes two numbers as input from the user and prints their product.
7. What is operator precedence? In the expression 3 + 4 * 2, which operation is performed first?
8. How can you convert a string input (like "25") into an integer?
9. For the **QuickCalc** artifact, write a line of code to get the first number from the user.
10. For the **QuickCalc** artifact, after getting two numbers as input (assume they are stored in val1 and val2), write the code to calculate and print their difference.
11. What are comparison operators? List three examples.
12. What is a Boolean value?
13. Write a script that asks for the user's age and prints True if they are 18 or older, and False otherwise.
14. Explain string concatenation and string repetition with examples.
15. In the **Headline Fetcher** artifact, if a headline is stored as a string, how could you add a prefix like "Breaking: " to it?

Assignment 3: Conditional Statements (If-Else)

1. What is the purpose of an if statement in Python?
2. Explain the syntax of an if-elif-else ladder.
3. Write a Python script that takes a number as input and prints "Positive" if it's greater than 0, "Negative" if less than 0, and "Zero" if it is 0.
4. What are logical operators (and, or, not)? Provide an example of how and can be used in an if statement.
5. In the **Number Guesser Game**, how would you use an if statement to check if the user's guess is correct?
6. Continuing with the **Number Guesser Game**, how would you use elif to tell the user if their guess is "too high"?
7. Write a script that asks for a password. If the password is "python123", print "Access granted", otherwise print "Access denied".
8. What is meant by "nesting" conditional statements? Provide a simple example.
9. For the **QuickCalc** artifact, how would you use an if-elif-else structure to perform

an operation (e.g., '+', '-', '*', '/') based on user input for the operator?
10. Write a script that checks if a number is even or odd.
11. What is the difference between == and =?
12. Can an if statement exist without an else block? Explain.
13. For the **GradeBook Analyzer**, write a condition to check if a student's score (e.g., score = 85) is an 'A' grade (e.g., score >= 90).
14. How can you check if a string input by the user is empty?
15. Write a script that asks for a number. If the number is between 1 and 10 (inclusive), print "In range". Otherwise, print "Out of range".

Assignment 4: Loops (For Loops)

1. What is a for loop in Python? When is it typically used?
2. Explain the range() function. What does range(5) generate? What about range(2, 7)? And range(1, 10, 2)?
3. Write a Python script using a for loop to print numbers from 1 to 10.
4. How can you iterate through the characters of a string using a for loop?
5. Given a list my_list = [10, 20, 30, 40], write a for loop to print each element.
6. In the **TaskMaster CLI**, if tasks are stored in a list called tasks, write a for loop to display each task with its number (e.g., "1. Buy milk", "2. Study Python").
7. Write a script to calculate the sum of numbers from 1 to 100 using a for loop.
8. What is a nested for loop? Give an example of printing a 3x3 grid of asterisks.
9. For the **Headline Fetcher**, if you have a list of headlines, write a for loop to print each headline.
10. How can you use a for loop to count the number of vowels in a given string?
11. Write a script that uses a for loop to print the multiplication table of 5 (up to 5 x 10).
12. Can you use an else block with a for loop in Python? If so, when does it execute?
13. For the **GradeBook Analyzer**, if you have a list of scores, write a for loop to find the highest score.
14. What happens if you try to modify a list while iterating over it with a for loop? Is it recommended?
15. Write a for loop that iterates 5 times. In each iteration, it should ask the user for a number and add it to a running total. Finally, print the total.

Assignment 5: Loops (While Loops)

1. What is a while loop in Python? How does it differ from a for loop?
2. Write the basic syntax of a while loop.
3. Write a Python script using a while loop to print numbers from 1 to 10.
4. What is an infinite loop? How can you accidentally create one, and how can you

stop it?
5. Explain the use of the break statement within a loop.
6. Explain the use of the continue statement within a loop.
7. In the **Number Guesser Game**, how would a while loop be used to allow the user to keep guessing until they get the right number?
8. Write a script that keeps asking the user for input until they type "quit".
9. For the **QuickCalc** artifact, how could a while loop be used to allow the user to perform multiple calculations until they choose to exit?
10. Write a script that uses a while loop to find the first number divisible by 7 in the range of 1 to 100.
11. What is a sentinel value in the context of while loops?
12. Can you use an else block with a while loop? If so, when does it execute?
13. Write a while loop that simulates a countdown from 10 to 1, then prints "Liftoff!".
14. For the **TaskMaster CLI**, imagine you want to limit the number of tasks to 5. How might a while loop and a counter be used when adding tasks?
15. Convert the following for loop to an equivalent while loop:
    for i in range(5):
        print(i)

Assignment 6: Data Structures - Lists

1. What is a list in Python? How do you create an empty list?
2. How do you access elements in a list? What is indexing?
3. What happens if you try to access an index that is out of bounds for a list?
4. Explain list slicing with an example. my_list[1:4]
5. List and briefly describe three common list methods (e.g., append, insert, remove).
6. For the TaskMaster CLI, show how to:
   a. Create an empty list called tasks.
   b. Add "Pay bills" to the tasks list.
   c. Add "Call mom" to the beginning of the tasks list.
7. How do you find the length of a list?
8. Write a script to create a list of 5 of your favorite fruits. Then, print the list.
9. How can you remove an element from a list by its value? By its index?
10. For the **GradeBook Analyzer**, if student scores are [88, 92, 75, 95, 80], how would you add a new score 89 to this list? How would you remove the score 75?
11. What does the sort() method do for a list of numbers? For a list of strings?
12. How can you check if an item exists in a list?
13. Can a list contain elements of different data types? Give an example.

14. What is the difference between list.append() and list.extend()?
15. For the **Headline Fetcher**, if headlines = ["News A", "News B", "News C"], how would you access and print the second headline?

Assignment 7: Data Structures - Tuples and Dictionaries

1. What is a tuple in Python? How is it different from a list?
2. How do you create a tuple? Can you change elements in a tuple after it's created?
3. When might you prefer to use a tuple over a list?
4. What is a dictionary in Python? What does it consist of?
5. How do you create an empty dictionary? How do you add a key-value pair to a dictionary?
6. For the **GradeBook Analyzer**, how could you store a student's record (e.g., name: "Alice", score: 90, grade: "A") using a dictionary?
7. How do you access a value in a dictionary given its key? What happens if the key doesn't exist?
8. How can you get all keys from a dictionary? How about all values?
9. Write a script to create a dictionary representing a phone book (name: number). Add three entries and print one.
10. For the **TaskMaster CLI**, instead of just a list of task strings, how could you use a list of dictionaries, where each dictionary represents a task with properties like description and priority? Show an example of one such task dictionary.
11. What does the items() method return for a dictionary? How can you iterate through key-value pairs using it?
12. How do you remove a key-value pair from a dictionary?
13. Can dictionary keys be of any data type? What about values?
14. Explain tuple packing and unpacking with an example.
15. If the **QuickCalc** artifact needed to store the count of how many times each operation (+, -, *, /) was performed, which data structure (list, tuple, or dictionary) would be most suitable and why?

Assignment 8: Functions - Basics

1. What is a function in Python? Why are functions useful in programming?
2. How do you define a function in Python? Use the def keyword and provide a simple example.
3. What is the difference between a function parameter and a function argument?
4. Write a function called greet that takes a name as a parameter and prints "Hello, [name]!". Call this function with your name.
5. What does the return statement do in a function? Can a function return multiple values?

6. Write a function add_numbers that takes two numbers as parameters and returns their sum.
7. For the **QuickCalc** artifact, define a function calculate_sum(num1, num2) that returns the sum of two numbers.
8. What is variable scope? Explain the difference between local and global variables.
9. Can a function call another function? Provide a conceptual example.
10. What is a docstring in a Python function? How do you write one?
11. Write a function is_even that takes a number and returns True if it's even, False otherwise.
12. For the **Number Guesser Game**, write a function check_guess(secret_number, user_guess) that prints "Correct!", "Too high!", or "Too low!" and returns True if correct, False otherwise.
13. What happens if a function doesn't have a return statement? What does it implicitly return?
14. Write a function get_first_element that takes a list and returns its first element. What should it do if the list is empty?
15. How can functions help in organizing the code for the **TaskMaster CLI** (e.g., functions for adding a task, viewing tasks, removing a task)?

Assignment 9: Functions - Advanced Concepts

1. What are default argument values in Python functions? Provide an example.
2. How do you call a function using keyword arguments? What is the benefit?
3. Explain *args in a function definition. What does it allow you to do?
4. Explain **kwargs in a function definition. What does it allow you to do?
5. Write a function print_info that takes a name and an age (with a default value of 20) and prints them. Show calls using positional and keyword arguments.
6. What is a lambda function (anonymous function)? Write a simple lambda function that adds two numbers.
7. When might you use a lambda function instead of a regular function defined with def?
8. Consider the **GradeBook Analyzer**. Write a function calculate_average(scores_list) that returns the average of scores in a list. Handle the case of an empty list (return 0).
9. Can a function be assigned to a variable in Python? Demonstrate.
10. Can functions be passed as arguments to other functions? Give a conceptual example.
11. What is recursion? Write a simple recursive function to calculate factorial (e.g., 5! = 5*4*3*2*1).
12. For the **QuickCalc** artifact, imagine you have separate functions for add(a,b),

subtract(a,b), etc. How could you use a dictionary to map operator symbols ('+', '-') to these functions?
13. What is the difference between a pure function and a function with side effects?
14. Write a function that uses *args to accept a variable number of numeric arguments and returns their sum.
15. How can you specify that a function parameter must be a keyword-only argument?

Assignment 10: String Manipulation

1. How do you find the length of a string?
2. Explain string slicing. Given s = "Python Programming", what is s[0:6]? s[7:]? s[::-1]?
3. List and describe three common string methods (e.g., upper(), lower(), strip()).
4. How can you check if a string starts with a particular substring? Ends with a particular substring?
5. What does the find() method return if the substring is found? If not found?
6. For the **Headline Fetcher**, if a headline is headline = " extra spaces around news ", how would you clean it to remove leading/trailing whitespace and capitalize the first letter?
7. What is the difference between find() and index() string methods?
8. How do you replace a part of a string with another string? Example: change "I like cats" to "I like dogs".
9. What does the split() method do? How can you split a string into words?
10. What does the join() method do? How can you join a list of words into a sentence?
11. For the **TaskMaster CLI**, if a user enters a task like "buy groceries", how can you ensure it's stored with the first letter capitalized (e.g., "Buy groceries")?
12. How can you check if all characters in a string are alphabetic? Numeric?
13. What are f-strings (formatted string literals) in Python? Provide an example.
14. Write a script that takes a sentence as input and counts the occurrences of the word "the" (case-insensitive).
15. In the **GradeBook Analyzer**, if student names are read as "lastname, firstname", how could you split this string to get the first and last names separately?

Assignment 11: File Handling - Reading Files

1. Why is file handling important in programming?
2. How do you open a file in Python for reading? What is the open() function?
3. What are file modes? What does 'r' mode signify?
4. Explain the importance of closing a file after use. What is the close() method?
5. What is the with open(...) as ...: statement, and why is it preferred for file

handling?
6. How do you read the entire content of a file into a single string? (method name)
7. How do you read a file line by line using a loop? (method name)
8. How do you read all lines of a file into a list of strings? (method name)
9. For the **GradeBook Analyzer**, if student data is in grades.txt where each line is "Name,Score" (e.g., "Alice,85"), outline the steps to read this file and print each student's name and score.
10. What happens if you try to open a file for reading that does not exist?
11. How can you check if a file exists before trying to open it? (Hint: use the os module).
12. What is a file path? Differentiate between an absolute and a relative path.
13. After reading a line from a file, why might you need to use strip() on it?
14. Write a Python script that reads the content of a file named story.txt and prints the total number of words in it. (Assume story.txt exists and has some text).
15. For the **Headline Fetcher**, if headlines were stored one per line in a file headlines_archive.txt, write code to read and print the first 5 headlines.

Assignment 12: File Handling - Writing Files

1. How do you open a file in Python for writing? What file mode is used? What happens if the file already exists?
2. What file mode is used to append content to an existing file? What happens if the file doesn't exist?
3. What is the write() method used for? Does it add a newline character automatically?
4. How can you write a list of strings to a file, with each string on a new line?
5. For the **TaskMaster CLI**, how would you save the current list of tasks (e.g., tasks = ["Buy milk", "Read book"]) to a file named mytasks.txt? Each task should be on a new line.
6. Write a script that creates a file named numbers.txt and writes numbers from 1 to 10, each on a new line.
7. What is the difference between 'w' and 'a' file modes?
8. How can you ensure that data written to a file is actually saved to disk immediately? (Hint: flush())
9. For the **GradeBook Analyzer**, if you calculate an average score, how would you write this average to a new file called summary.txt?
10. Is it possible to read from and write to a file simultaneously? What modes might be used (e.g., 'r+', 'w+')?
11. Write a script that prompts the user for their name and then appends "[timestamp] - [name] logged in" to a file called activity_log.txt. (You can use a

placeholder for the timestamp).

12. What considerations should you have regarding file permissions when writing files?
13. If you are writing non-string data (like numbers) to a file, what must you do first?
14. For the **Headline Fetcher**, if you wanted to save a list of fetched headlines to a file for later use, describe the steps and file mode you would use.
15. Create a simple logger function that takes a message string and appends it with a newline to app.log.

Assignment 13: Error Handling (Try-Except)

1. What is an exception in Python? Give an example of a common exception (e.g., ZeroDivisionError, TypeError).
2. What is the purpose of a try-except block?
3. Write the basic syntax of a try-except block.
4. How can you handle a specific type of exception? Example: ValueError.
5. What happens if an exception occurs inside a try block and there is no matching except block?
6. For the **QuickCalc** artifact, how would you use a try-except block to handle potential ValueError if the user enters text instead of a number for an operand?
7. How can you handle multiple types of exceptions with a single except block?
8. What is the purpose of the else clause in a try-except block? When does it execute?
9. What is the purpose of the finally clause in a try-except block? When does it execute?
10. For the **GradeBook Analyzer**, when reading scores from a file, what kind of error might occur if a score is not a valid number? How would you handle it?
11. How can you get access to the exception object itself within an except block?
12. Is it a good practice to use a bare except: clause? Why or why not?
13. Write a script that attempts to divide two numbers input by the user. Handle ZeroDivisionError and ValueError appropriately.
14. For the **File Handling** assignments, what exception should you specifically try to catch if a file you are trying to open for reading does not exist? (FileNotFoundError)
15. How can you raise an exception manually using the raise keyword? Give an example.

Assignment 14: Modules and Libraries

1. What is a module in Python? Why are modules useful?
2. How do you import a module in Python? Show two ways: import module_name

and from module_name import specific_function.

3. What is the Python Standard Library? Give two examples of modules included in it.
4. How can you use the math module to calculate the square root of a number?
5. How can you use the random module to generate a random integer between 1 and 100 (inclusive)?
6. For the **Number Guesser Game**, which module would be essential for the computer to pick a secret number? Show the import statement and the function call.
7. What is the purpose of the as keyword when importing modules (e.g., import random as rd)?
8. How can you find out what functions and attributes are available in an imported module? (Hint: dir() or help()).
9. If you have your own Python file named my_utils.py with a function helper(), how would you import and use helper() in another Python script in the same directory?
10. What is PIP? What is it used for?
11. For the **Headline Fetcher** (if it were a real web scraper), what popular third-party library might be used for making HTTP requests? (e.g., requests). You don't need to write code, just name it.
12. How can the datetime module be used to get the current date and time?
13. Write a script that imports the os module and uses it to list all files and directories in the current working directory.
14. For the **GradeBook Analyzer**, if you wanted to calculate the median score, you might look for a statistics-related module. Name one such module in the Python Standard Library. (statistics)
15. What is the difference between importing a whole module (import math) versus importing specific names from it (from math import sqrt) in terms of how you call the functions?

Assignment 15: Putting It All Together - Mini-Projects

1. QuickCalc Enhancement:
   Design and write a Python script for the QuickCalc artifact. It should:
   a. Ask the user for two numbers.
   b. Ask the user for an operation (+, -, *, /).
   c. Perform the calculation and print the result.
   d. Include error handling for invalid number input and division by zero.
   e. (Bonus) Allow the user to perform multiple calculations until they choose to exit.

2. Number Guesser Game - Core Logic:
   Write the core logic for the Number Guesser Game.
   a. Generate a random secret number between 1 and 100.
   b. Use a while loop to let the user guess.
   c. Provide feedback ("Too high", "Too low", "Correct!").
   d. Count the number of guesses.
3. TaskMaster CLI - Add & View:
   Write a script for the TaskMaster CLI that allows users to:
   a. Add a new task to a list.
   b. View all current tasks.
   c. Implement this using functions for adding and viewing.
   (Do not worry about saving to a file for this question).
4. GradeBook Analyzer - Basic Stats:
   Given a list of scores: scores = [85, 92, 78, 95, 88, 70, 90, 82]
   Write a script to calculate and print:
   a. The average score.
   b. The highest score.
   c. The lowest score.
   Use functions for each calculation.
5. Headline Formatter:
   For the Headline Fetcher concept, write a function that takes a list of headline strings. The function should return a new list where each headline is title-cased (e.g., "this is a headline" becomes "This Is A Headline") and has any leading/trailing whitespace removed.
6. Define a function that simulates rolling two six-sided dice. It should return the sum of the two dice.
7. Write a script that reads text from a file named input.txt, counts the frequency of each word (case-insensitive), and prints the word counts. (Assume input.txt exists).
8. Create a dictionary to store information about 3 different books (title, author, year). Write a function to print the details of a book given its title.
9. Write a function that takes a list of numbers and returns a new list containing only the even numbers from the original list.
10. Implement a simple password validator function. It should take a password string and return True if it's at least 8 characters long and contains at least one digit, False otherwise.
11. Write a script that asks the user for a sentence and then prints the sentence in reverse order (e.g., "hello world" becomes "dlrow olleh").
12. Using the datetime module, write a script to display the current date and time in

the format "YYYY-MM-DD HH:MM:SS".

13. TaskMaster CLI - Add with File Persistence:
    Extend question 3 (TaskMaster CLI). Modify the "add task" functionality so that after a task is added to the list, the entire list of tasks is saved to tasks.txt, with each task on a new line.
14. Number Guesser Game - Limited Guesses:
    Modify your Number Guesser Game from question 2 to give the user a limited number of guesses (e.g., 7). If they don't guess within the limit, the game ends, and the secret number is revealed.
15. GradeBook Analyzer - Reading from File:
    Modify your GradeBook Analyzer from question 4. Instead of a hardcoded list, read student scores from a file named scores.txt. Assume each line in scores.txt contains one score (as a number). Handle potential errors if the file doesn't exist or contains non-numeric data.